

SMART CREDIT AI

Real-Time Machine Learning Credit Risk Engine

A state-of-the-art intelligent platform combining predictive machine learning models, local/global Explainable AI (SHAP), automated database integrations, and Large Language Model insights to deliver institutional-grade credit risk assessments.

PROJECT DOCUMENTATION & USER MANUAL

Author: Chan Jin Wei

Workspace: Georaldo/credit-risk

Date: July 2026

1. System Overview & Core Capabilities

Smart Credit AI is a comprehensive modern web application designed to optimize and automate the evaluation of loan applicants. By utilizing advanced machine learning techniques alongside natural language reasoning, the system empowers financial institutions to assess applicant profiles rapidly, reduce credit default risk, and visualize data-driven justifications for every underwriting decision.

Core System Capabilities:

- **Automated Risk Assessment:** Instantly evaluates applicant profiles using a cost-sensitive RandomForestClassifier machine learning model. The model computes real-time default probabilities and segments applicants into clear risk levels (Low, Medium, High).
- **Local & Global Explainable AI (XAI):** Employs SHAP (SHapley Additive exPlanations) values to provide model transparency. Users can see local feature contributions (waterfall explanations) for individual loan decisions, and view a global feature importance beeswarm plot representing general model patterns.
- **Generative AI Executive Insights:** Connects to the Google Gemini 2.5 Flash API to automatically generate highly contextualized executive summaries. This helps bank managers understand model behavior, portfolio quality distribution, and provides strategic recommendations based on current applicants.
- **Oracle Database Integration:** Built-in session pooling to store and synchronize applicant details, loan request parameters, and final model decisions across tables like APPLICANTS, LOAN_REQUESTS, and PREDICTION_RESULTS.

2. System Architecture & Tech Stack

Component	Technology / Library Description
Frontend UI Framework	React 19, Vite (development server), Tailwind CSS (aesthetics)
Frontend Visualizations	Recharts (interactive dashboard charts), Lucide React (UI icons)
Backend Framework	FastAPI (Asynchronous Python web API with automated swagger/docs)
Machine Learning	Scikit-Learn (RandomForestClassifier, ColumnTransformer, Pipeline)
Explainable AI (XAI)	SHAP (TreeExplainer for tree-based ensemble explanation and SHAP plot)
Generative AI Agent	Google Gemini 2.5 Flash API (REST API direct request integrations)
Database Backend	Oracle Database (Express Edition XE/Enterprise) with cx_Oracle pooling
Execution & Tooling	Windows PowerShell runner script, python-dotenv (configurations)

Database Architecture Schema:

The database integration uses a relational Oracle schema representing standard loan origination data flows:

1. **APPLICANTS** Table: Stores applicant personal info (name, age, income, employment length, credit history length).
2. **LOAN_REQUESTS** Table: Tracks loan details (amount, intent category, interest rate, grade classification).
3. **PREDICTION_RESULTS** Table: Records model output probability, final risk level (Low/Medium/High), and underwriting decision status.

3. Operations Manual - How to Run

A. Environment & Credentials Configuration

Before launching the system, configure the system variables inside the root .env file. The file should include your Google Gemini API key:

```
GEMINI_API_KEY=AIzaSy...
```

If using Oracle Database, ensure your credentials (host, port, username, password, service) are correctly set up or exported in your session. By default, the application runs on a local CSV fallback database if connection attempt fails.

B. Running via the Automated Startup Script (Recommended)

An automation script (run_project.ps1) has been created to launch both client and server automatically:

1. Open Windows PowerShell in the project directory:

```
cd path\to\credit-risk
```

2. Run the script:

```
.\run_project.ps1
```

The script automatically detects the 'oracle_api_env' Conda environment (or configures a fallback Python venv), launches the FastAPI backend on `http://localhost:8000`, installs React dependencies, and starts the Vite frontend on `http://localhost:5173`.

C. Troubleshooting & Manual Execution

- Backend Manual Start:

Activate your python environment (oracle_api_env) and run:

```
cd backend
```

```
python main.py
```

- Frontend Manual Start:

```
cd frontend
```

```
npm install
```

```
npm run dev
```